Introduction to Algorithms and Data Structures

week 5: Lists

SAXION UNIVERSITY OF APPLIED SCIENCES

Deepdive into lists

- **Discuss work last lesson**
- The list interface
- Example list: Stack
- Linked list datastructure – add and remove
- Exercises / Homework

- Where you able to do the exercises?
  1. Yes! All exercises are done!
  2. No.., Because I have these questions
  3. I haven't prioritized this subject enough at the moment to do the assignments

Deepdive into lists

- **Discuss work last lesson**
- **The list interface**
- Example list: Stack
- Linked list datastructure – add and remove
- Exercises / Homework

List interface

- List
  - Sequence of elements
  - All accessible
  - Additions and removals at any point in the list
  - Methods: `isEmpty, insert, delete, find, …`
- (List) Iterator
  - Sequential visit of all the elements (enhanced for!)
- **<u>Implementations</u>:**

<u>Statically/fixed size</u>	<u>Dynamically/flexible size)</u>
Array	ArrayList
	Linked List
	Doubly Linked List

- …

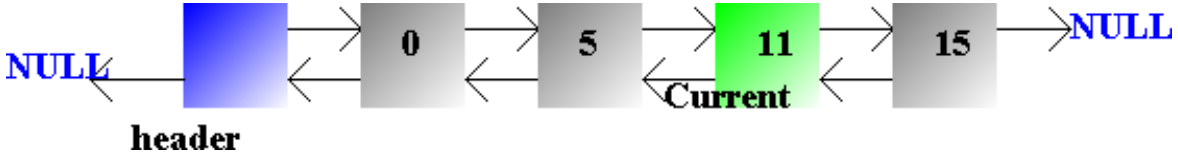
Implementations of List

- Example: array

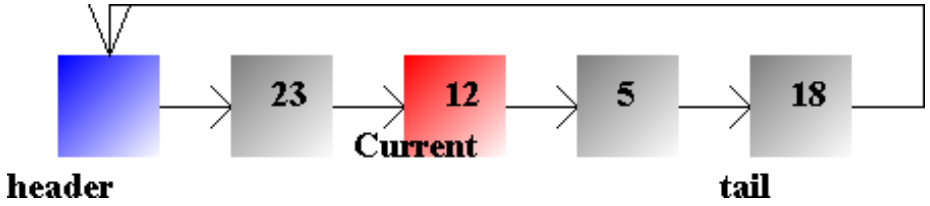
0	1	2	3	4	5	6	7
2	18	7	12	3	6	11	9

insert and delete
can be slow
… why?
… when not?

- Example: doubly linked list



- Example: circular linked list



Exercise: 1_arraylist

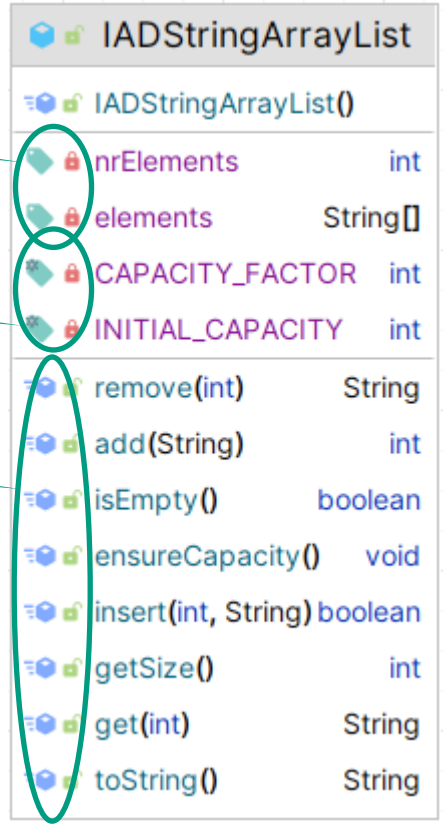
- **Task:**
  Implement our own version of the ArrayList in `MyStringArrayList` class
- Test your implementation by running th
  `main` method in `MyArrayListMain`

- Implementation advantages?
- Implementation downsides?

private instance variables

private class constants

public instance methods



Deepdive into lists

- **Discuss work last lesson**
- **The list interface**
- **Example list: Stack**
- Linked list datastructure – add and remove
- Exercises / Homework

Example Linked List: Stack

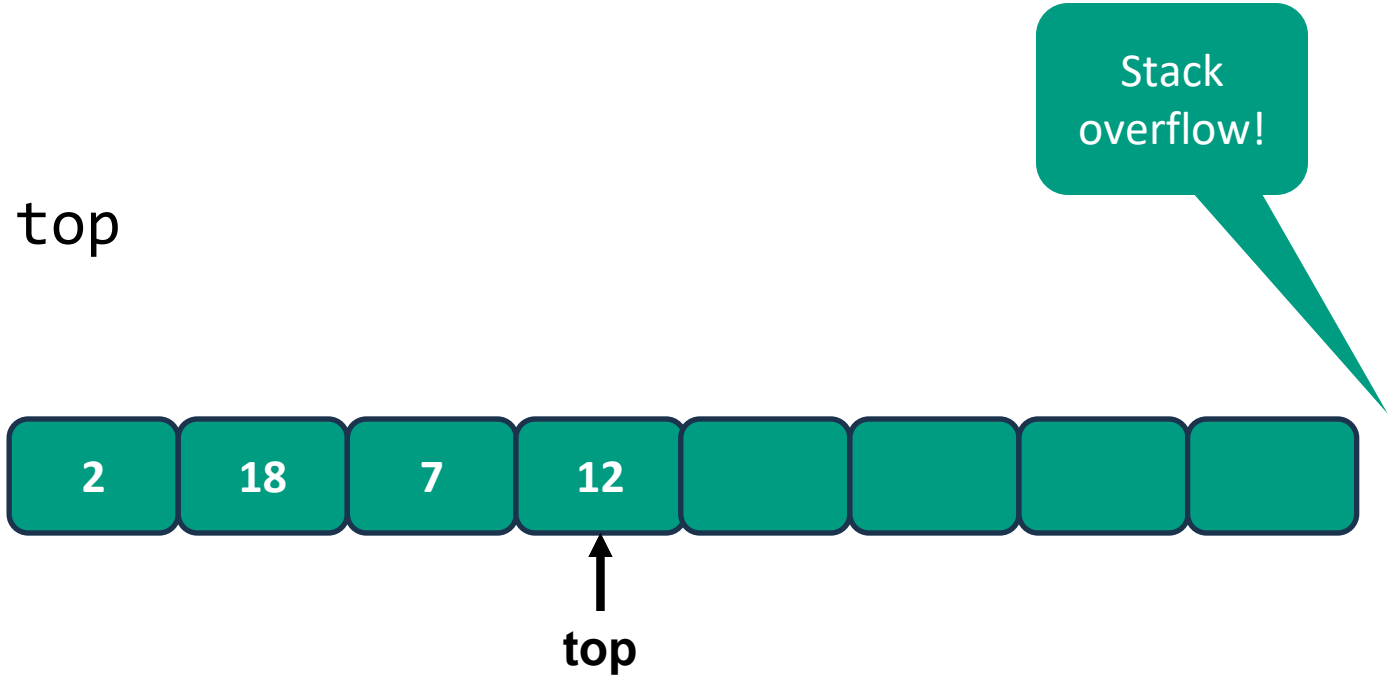
- Stack
  - Sequence of elements
  - **Last In First Out (LIFO)**
  - (only!) top is (should be) accessible
  - Key methods: *isEmpty, push, pop, peek*, …
- Implementations
  - Array – based stack
  - ArrayList – based stack
  - Linked List – based stack
  - …

Array implementation of Stack

- Example: (fixed) array:

```
Stack stack = new Stack();
stack.push(2);
stack.push(18);
stack.push(7);
stack.push(12); //current top
```

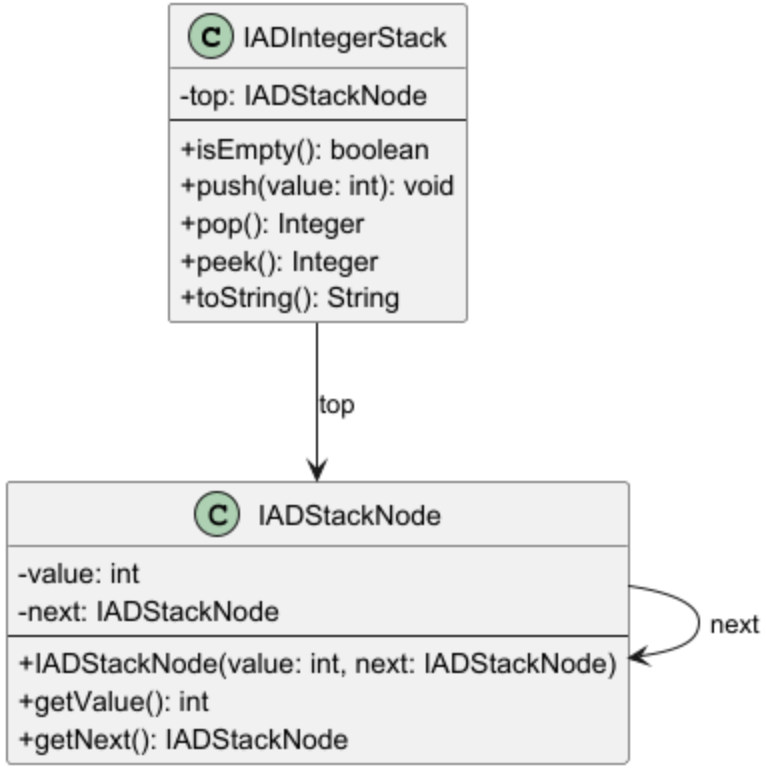
- Advantage(s)?
- Disadvantage(s)?

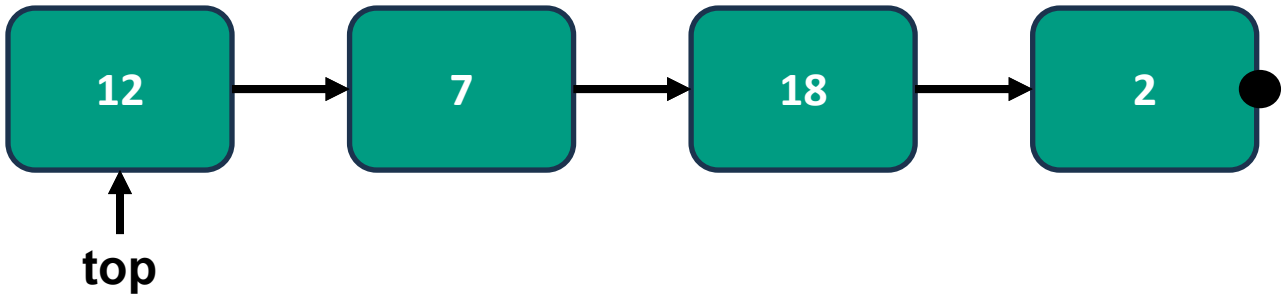


List implementation of Stack

```
Stack stack = new Stack();
stack.push(2);
stack.push(18);
stack.push(7);
stack.push(12); //current top
```

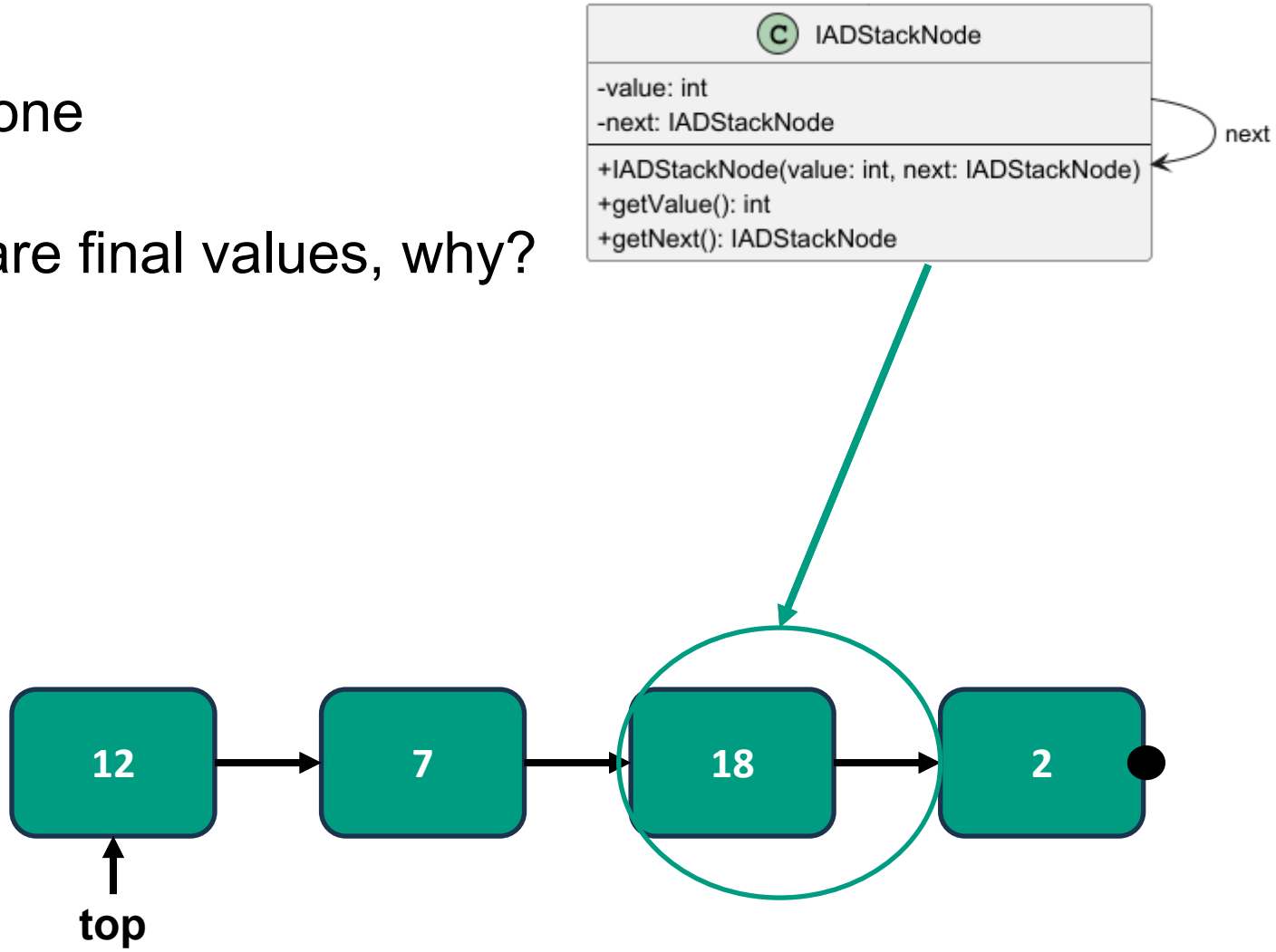
- Advantage(s)?
- Disadvantage(s)?





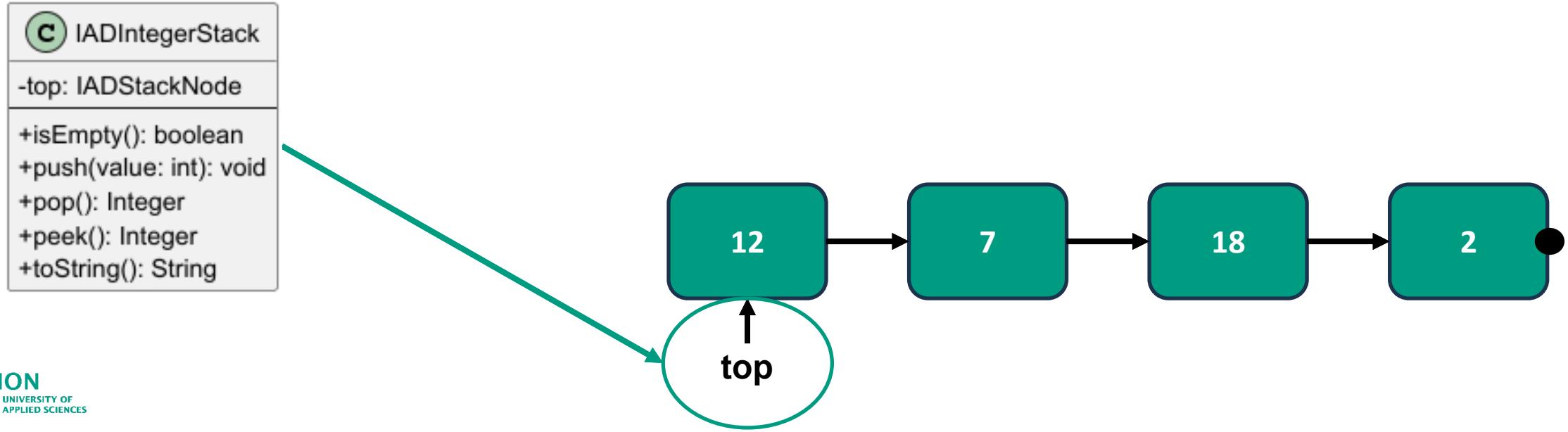
List implementation of Stack

- Every node points to the next one (or null if it's the last)
- The value and next variables are final values, why?



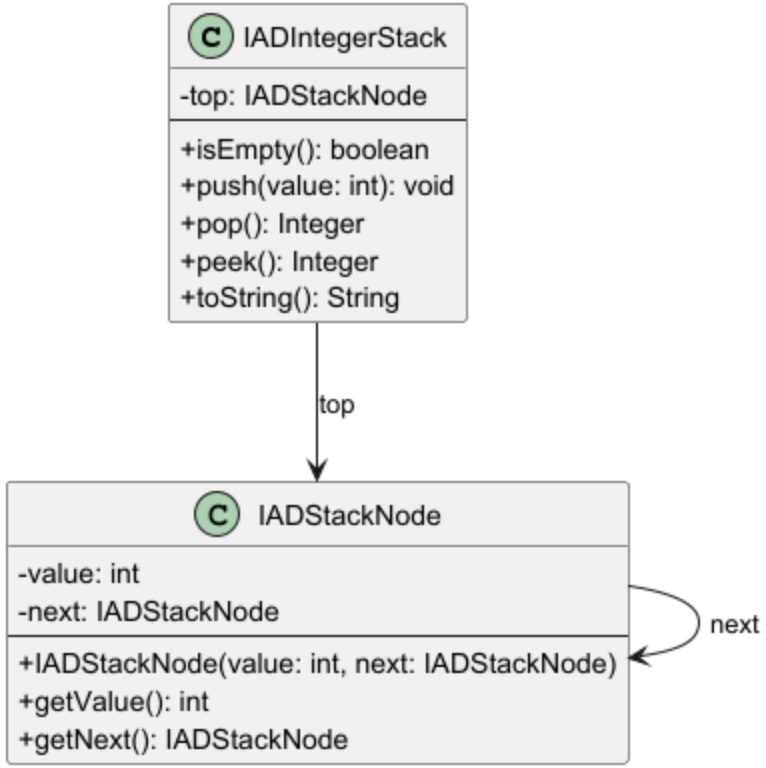
List implementation of Stack

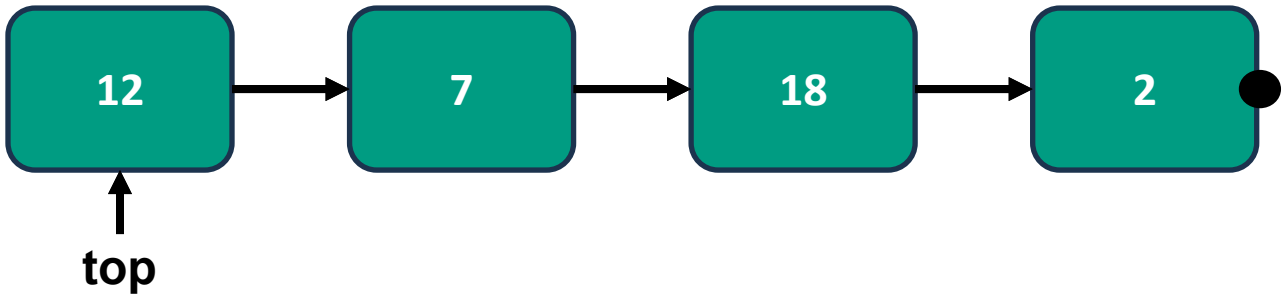
- Integer return values, not int, why?



List implementation of Stack

- See module 0_stack for the implementation



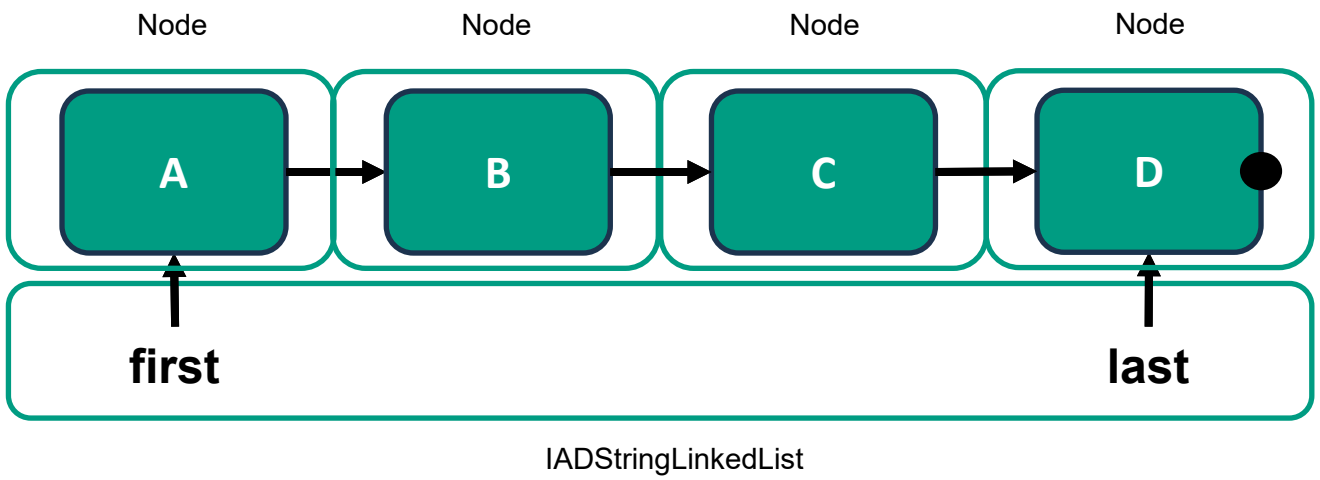


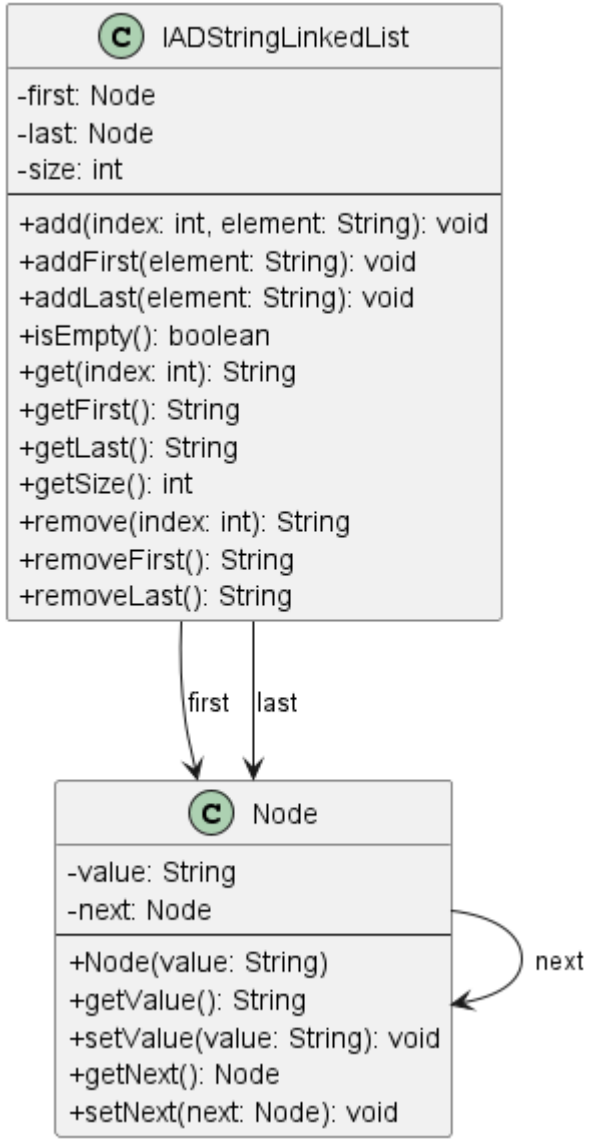
Deepdive into lists

- **Discuss work last lesson**
- **The list interface**
- **Example list: Stack**
- **Linked list datastructure – add and remove**
- Exercises / Homework

Singly linked list implementation (for String values)

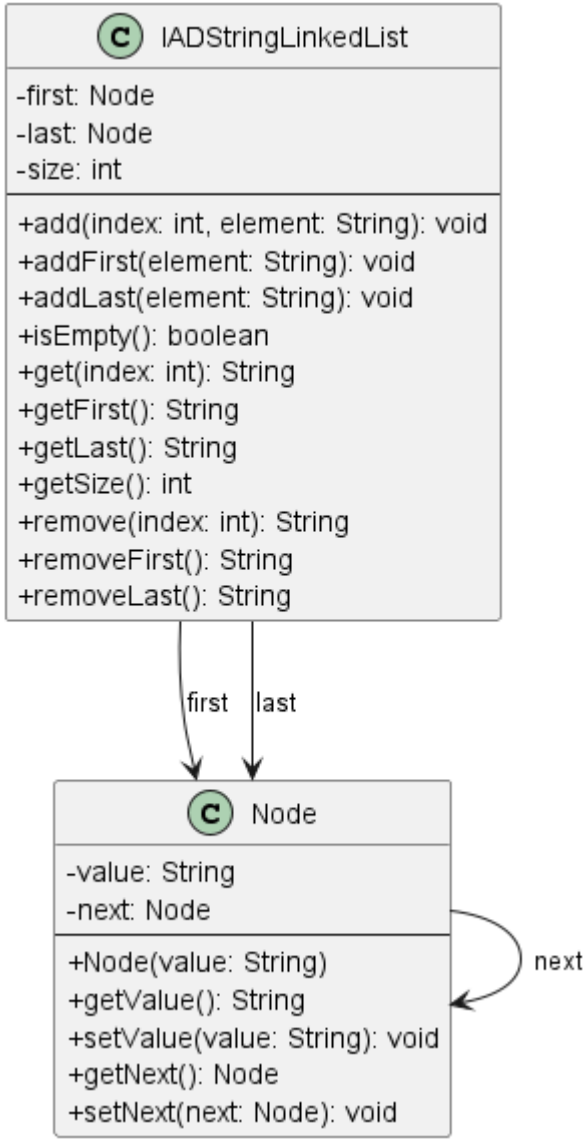
- IADStringLinkedList (see 2_remove in exercises)





Create the list

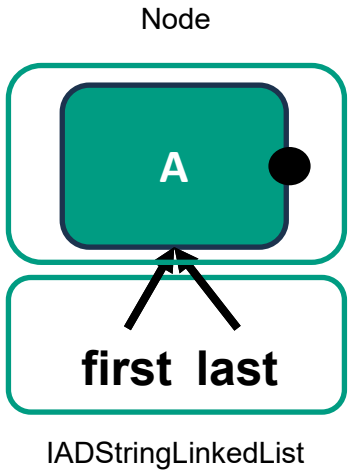
1. IADStringLinkedList list = new IADStringLinkedList();

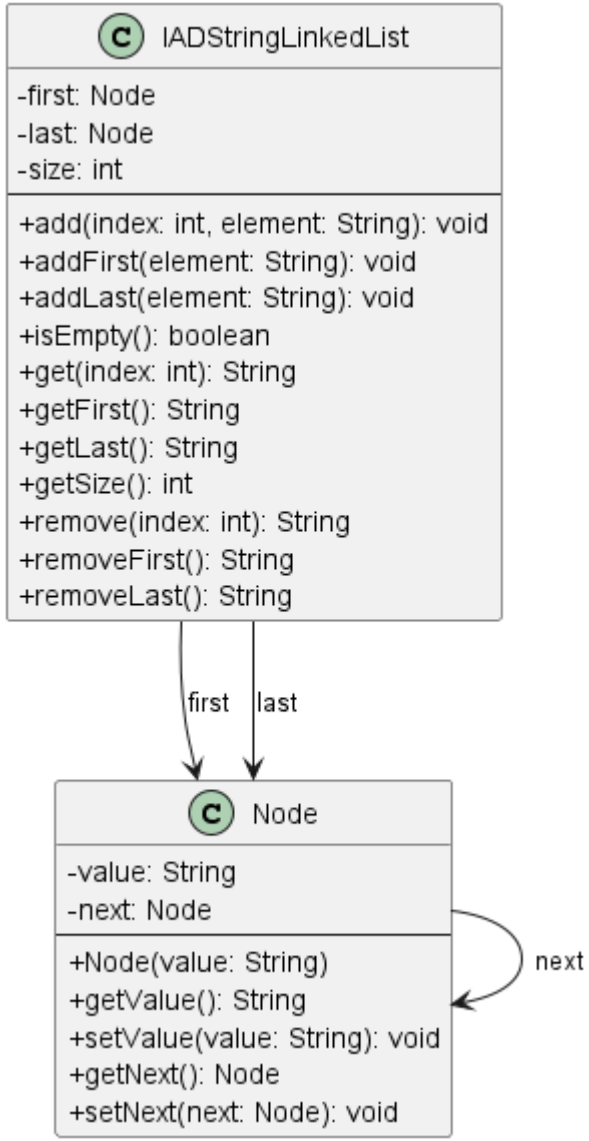


IADStringLinkedList

Add a node at the beginning

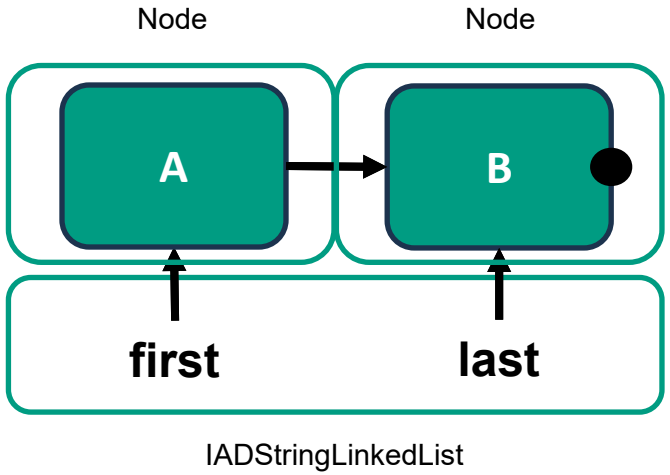
1. IADStringLinkedList list = new IADStringLinkedList();
2. list.addFirst("A");

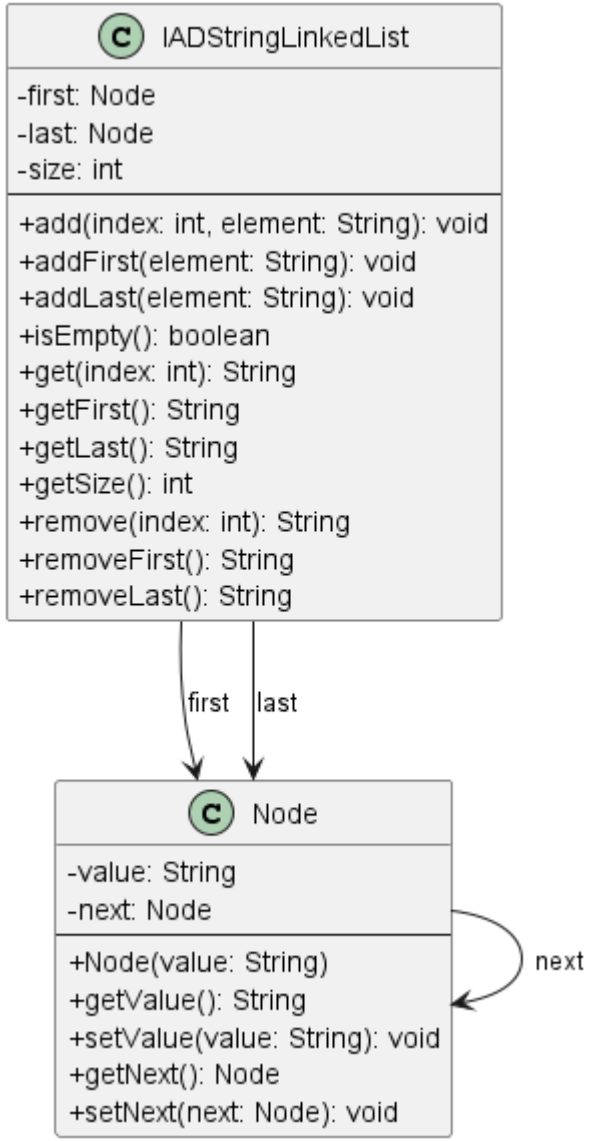




Add a node at the end

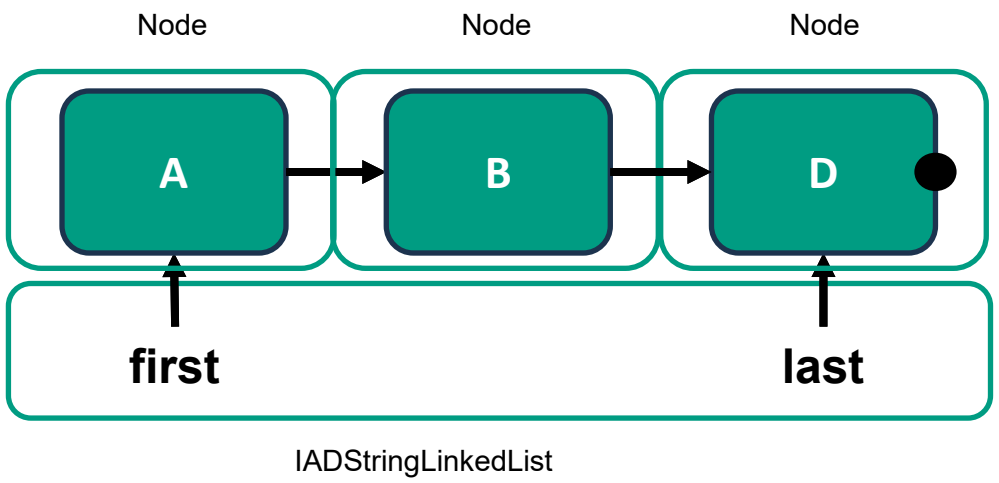
1. IADStringLinkedList list = new IADStringLinkedList();
2. list.addFirst("A");
3. list.addLast("B");

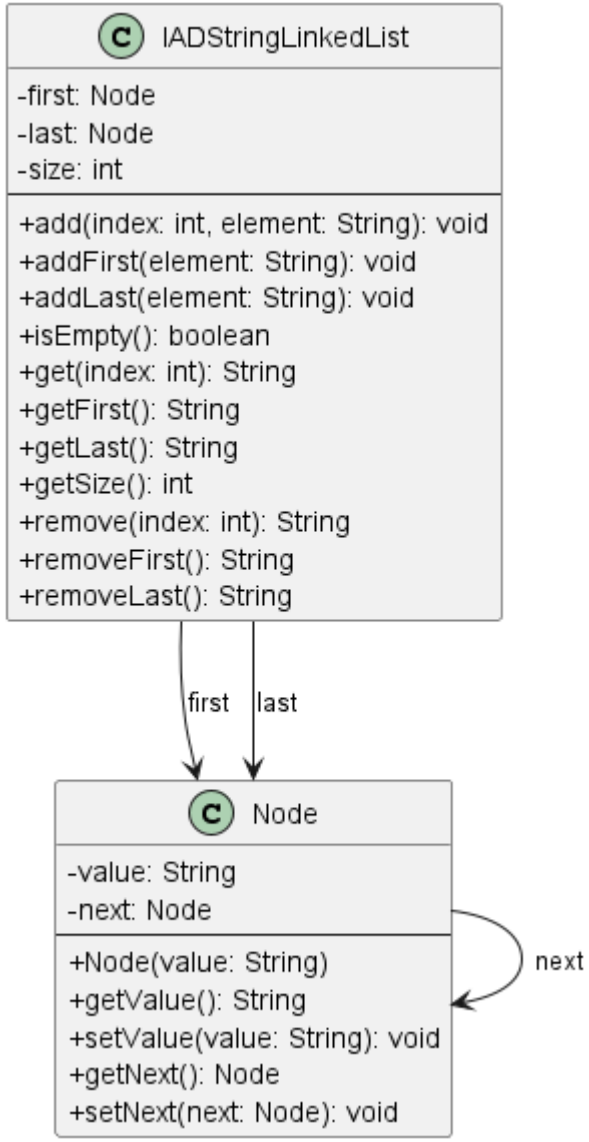




Add another node at the end

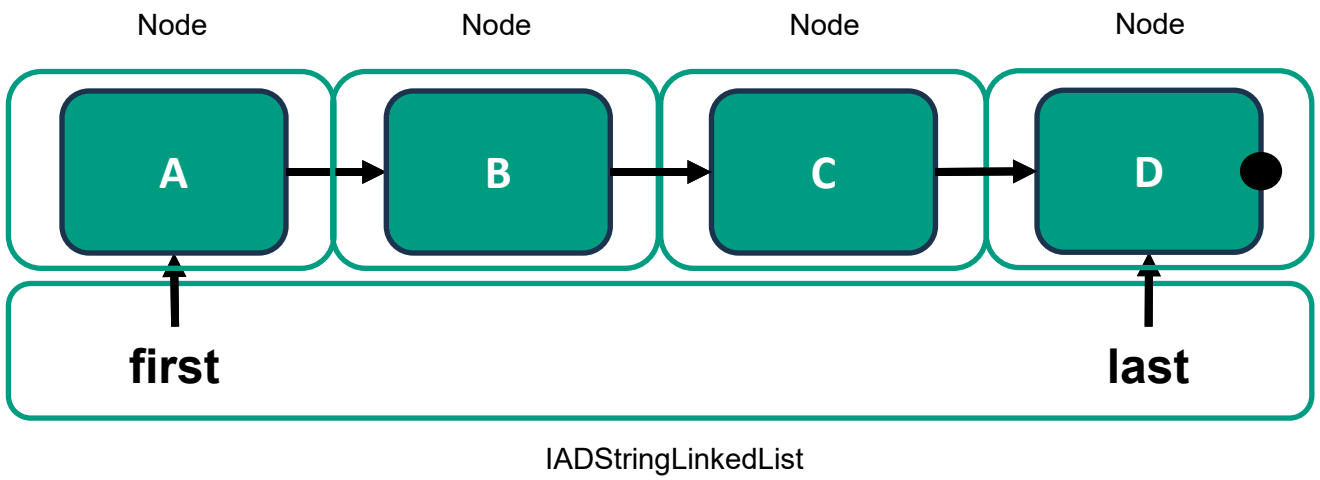
```
1.  IADStringLinkedList list = new IADStringLinkedList();
2.  list.addFirst("A");
3.  list.addLast("B");
4.  list.addLast("D");
```

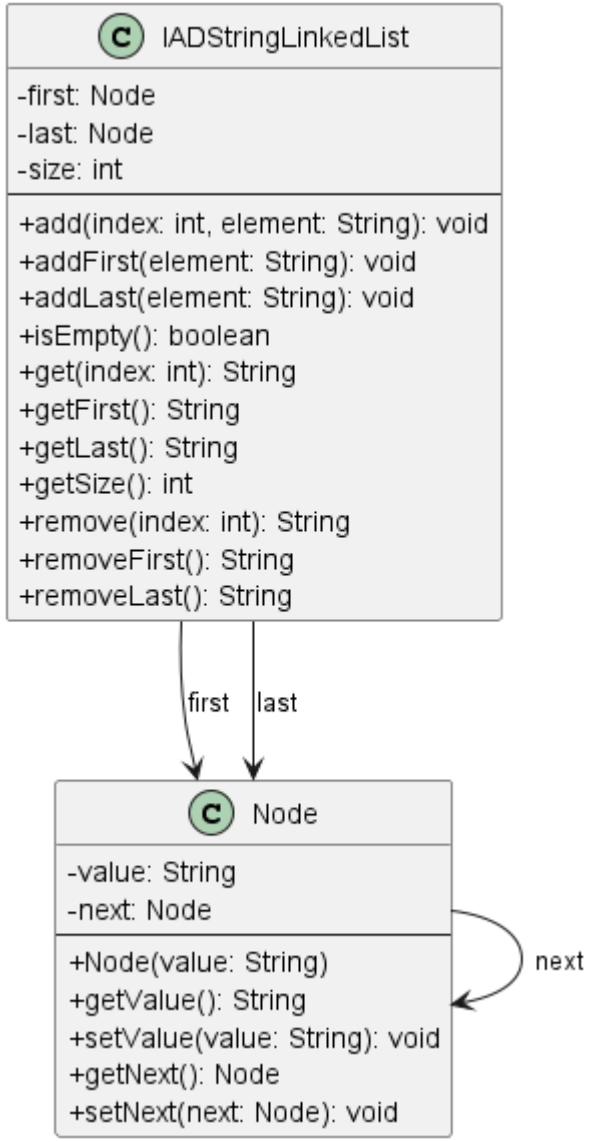




Adding a node on a specific position

```
1.  IADStringLinkedList list = new IADStringLinkedList();
2.  list.addFirst("A");
3.  list.addLast("B");
4.  list.addLast("D");
5.  list.add(2, "C");
```

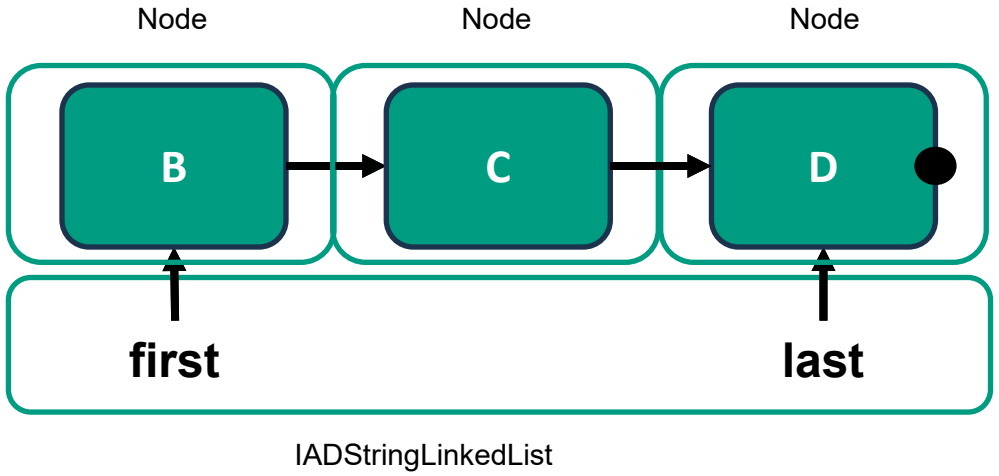


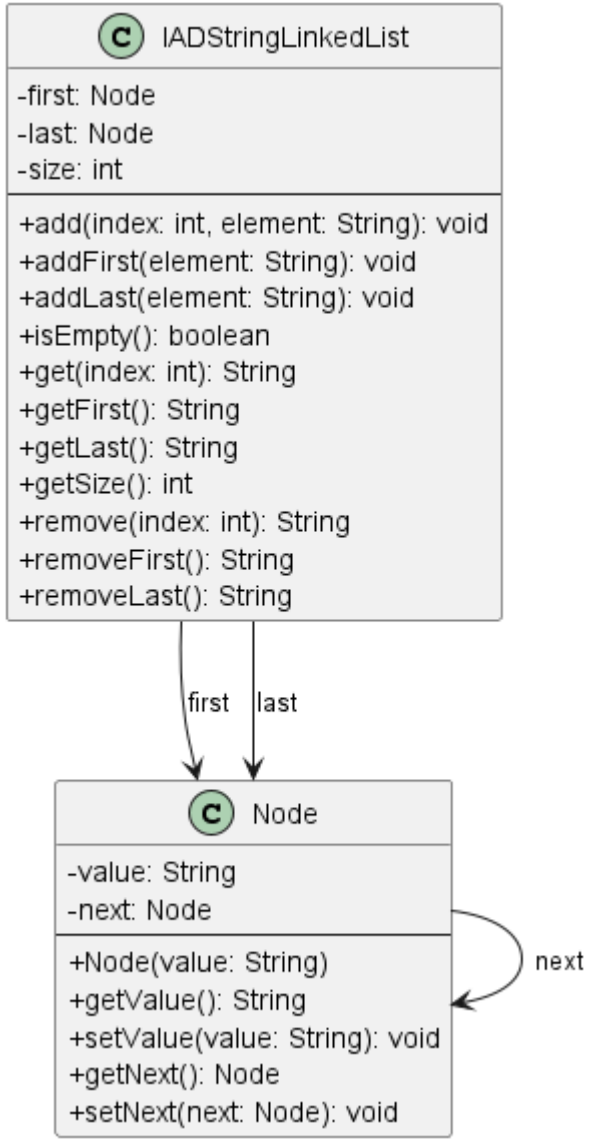


Removing the first node

(list == "[A, B, C, D]“)

1. list.removeFirst();

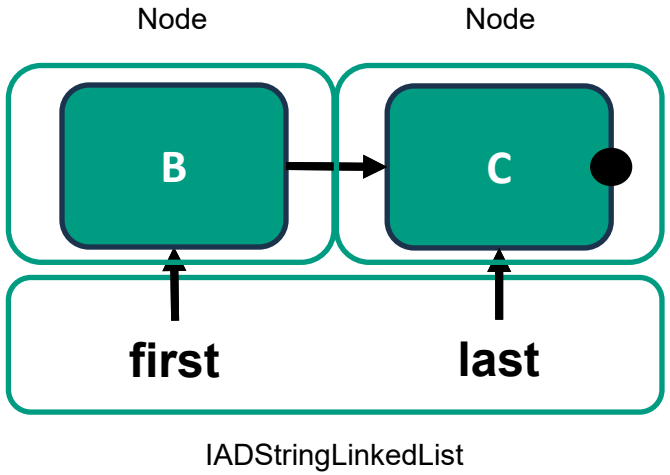


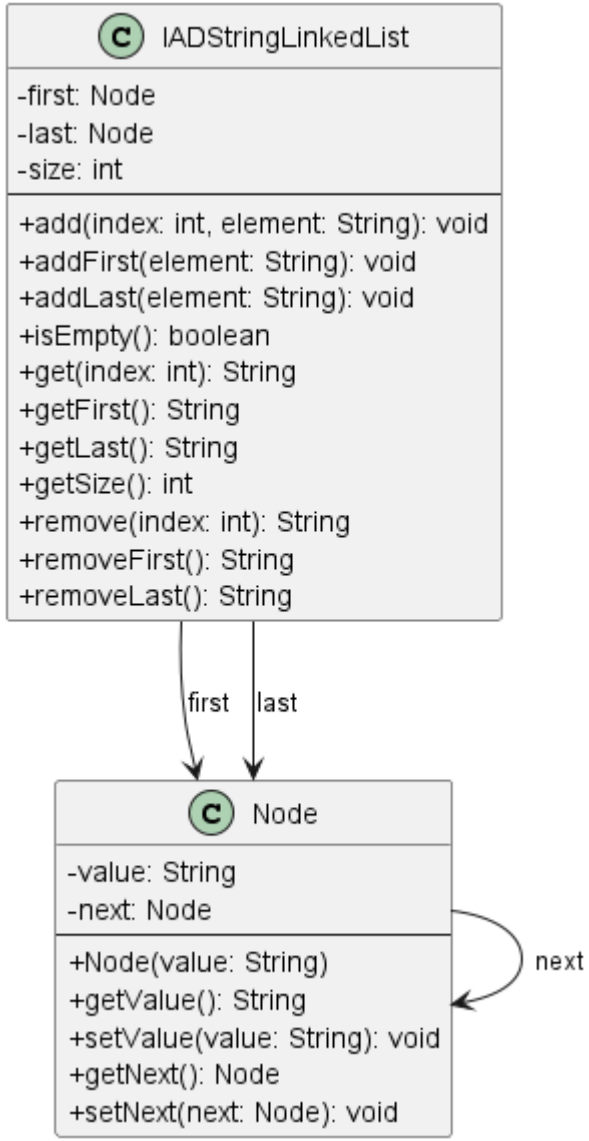


Removing the last node

(list == "[A, B, C, D]“)

1. list.removeFirst();
2. list.removeLast();

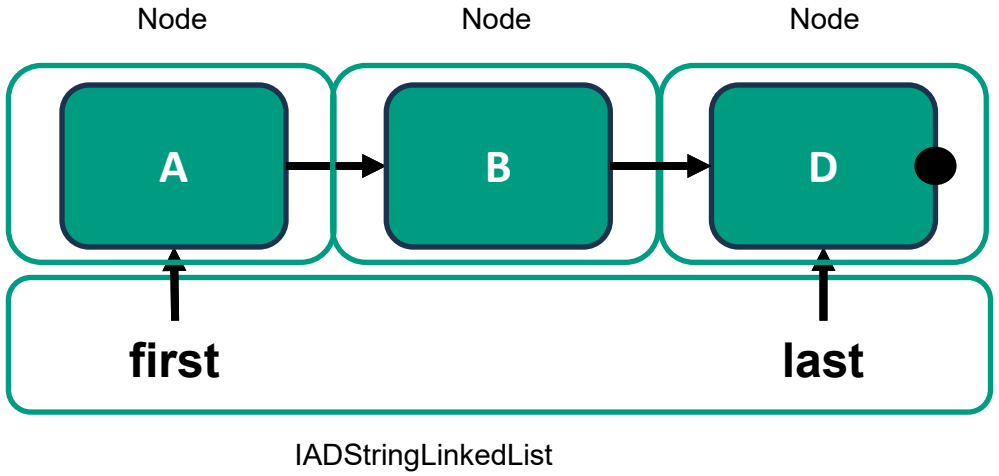


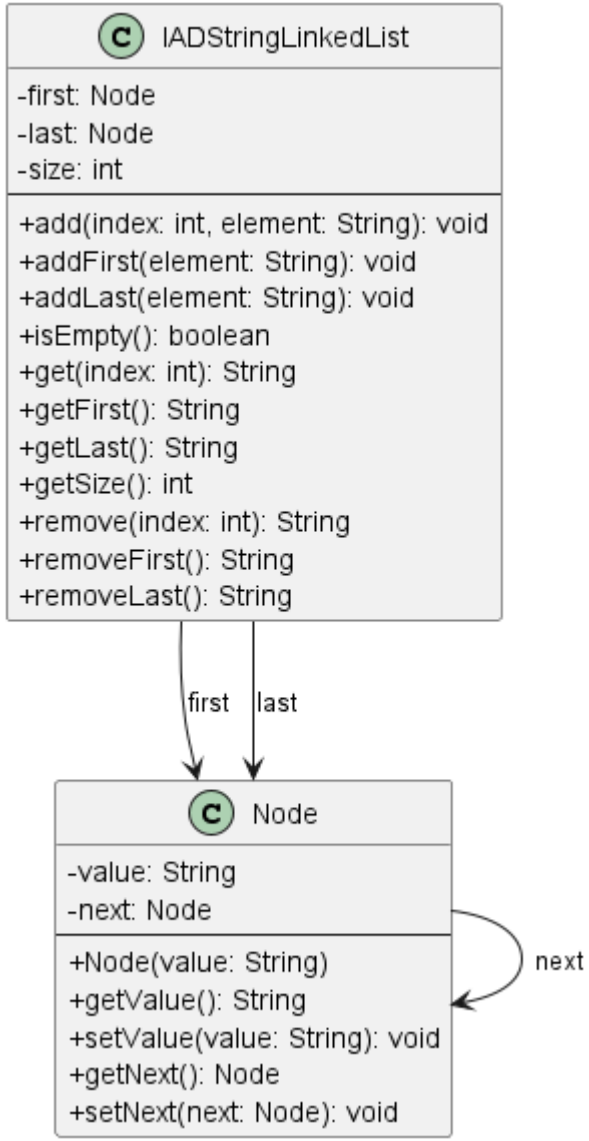


Removing the third node

(list == "[A, B, C, D]“)

1. list.remove(2);





Deepdive into lists

- Discuss work last lesson
- The list interface
- Example list: Stack
- Linked list datastructure – add and remove
- Exercises / Homework

Now it’s your turn!